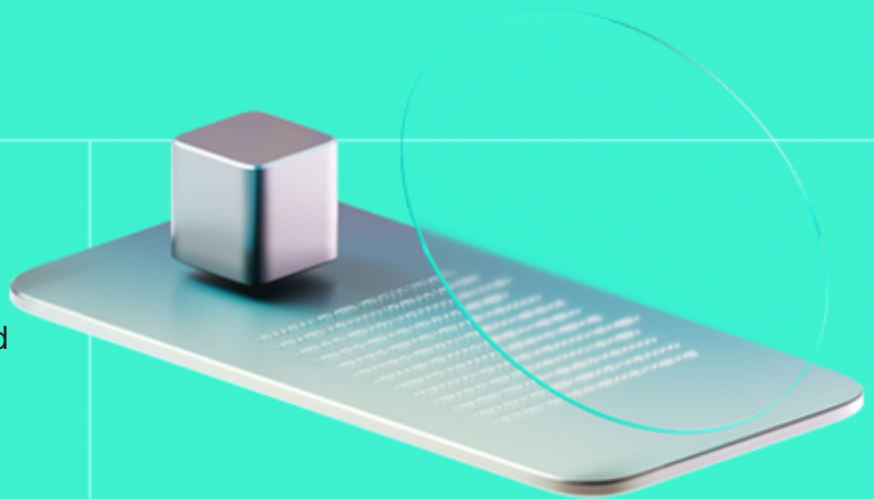




Smart Contract Code Review And Security Analysis Report

Customer: NEXPACE Limited

Date: 15/09/2025



We express our gratitude to the NEXPACE Limited team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

A modular smart contract framework for managing creators and their dApps, distributing rewards, and issuing NFT items through an NXPC-native token economy with on-chain supply tracking and NESO (ERC-20) conversion.

Document

Name	Smart Contract Code Review and Security Analysis Report for NEXPACE Limited
Audited By	Viktor Lavrenenko , Kaan Caglan
Approved By	Ivan Bondar
Website	https://www.nexon.com/Home/Game
Changelog	04/09/2025 - Preliminary Report 15/09/2025 - Final Report
Platform	EVM
Language	Solidity
Tags	ERC721, ERC20, Proxies
Methodology	https://hacken.io/cc/sc_methodology

Review Scope

Repository1	https://github.com/NEXPACE-Limited/nexpace-contracts/
Commit	acf6c48
Remediation Commit For Repository1	2e8cd49
Repository2	https://github.com/NEXPACE-Limited/maple-contract
Commit	acfc642

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

16

Total Findings

1

Resolved

15

Accepted

0

Mitigated

Findings by Severity

Severity	Count
Critical	0
High	0
Medium	3
Low	2

Vulnerability	Severity
F-2025-12421 - Missing Price Validation in `confirmRequest`	Medium
F-2025-12422 - Risk of Locked NXPC in `requestItemIssuance`	Medium
F-2025-12553 - Unrestricted Owner Authority in retrieveNeso() Enables Direct Token Transfers	Medium
F-2025-12424 - Unsafe and Centralized Blackhole Handling	Low
F-2025-12490 - Missing Validation for exchangeRate Can Lead To DOS	Low
F-2025-12423 - Misleading Naming of `_dAppOwner`	Info
F-2025-12425 - Misleading Error Message in confirmRequest	Info
F-2025-12426 - Redundant ERC20 Transfer Logic	Info
F-2025-12452 - Missing Existence Guards	Info
F-2025-12486 - Excessive Revert String Length Increases Gas Costs and Deployment Size	Info
F-2025-12585 - Missing Validation for poolAmount > 0 in requestItemIssuance() Allows Zero ItemAmount Requests	Info
F-2025-12615 - Missing Events For Critical Data	Info
F-2025-12623 - Manipulation in expectAmount() Can Mislead Off-Chain Pricing Logic	Info
F-2025-12633 - Incomplete Function Logic Leads to Discrepancy Between Implementation and Developer Intentions	Info
F-2025-12639 - Checks-Effects-Interactions Pattern Violation	Info
F-2025-12641 - Missing Address Zero Validation	Info

Documentation quality

- Functional requirements are partially missed:
 - The project's purpose is not described.
 - Business logic is not provided: the economic model of the `NXPAmountManager` is not described.
 - Use cases are partially provided.
 - Project's features are described.
- Technical description is not provided:
 - Key function descriptions are provided for the files in the scope.
 - Architectural overview is provided.
 - Roles and authorization are provided.
 - Information on used technologies is provided.

Code quality

- Best practices are not followed.
- The development environment is configured.

Test coverage

- Code coverage of the contracts from the [repo1](#) and [repo2](#) is **100%** (branch coverage).
 - Deployment and basic user interactions are covered with tests.
 - Negative cases coverage is provided.
 - Interactions by several users are tested thoroughly.

Table of Contents

System Overview	6
Privileged Roles	6
Potential Risks	9
Findings	10
Vulnerability Details	10
Disclaimers	42
Appendix 1. Definitions	43
Severities	43
Potential Risks	43
Appendix 2. Scope	44
Appendix 3. Additional Valuables	45

System Overview

Nexpace is a modular smart contract framework that enables the onboarding of creators and their dApps, facilitates secure reward distribution, and manages NFT item issuance through a NXPC-native token economy, complete with on-chain supply tracking and seamless conversion between native NXPC and the NESO ERC-20 token. It has the following smart-contracts:

- **CreatorFactory.sol**
Manages creators and dApps: creates creator multisig wallets, registers dApps, tracks relationships, and coordinates reward allocation.
- **ItemIssuance.sol**
Central item issuance manager: maintains universes and item pools, accepts issuance requests that burn NXPC, and authorizes confirmations and item registration.
- **CreatorWalletLogicUpgradeableV2.sol**
Creator-wallet business logic (upgradeable): defines functions creators can call via their multisig wallet (allocate/withdraw assets, request issuance, transfer NXPC, etc.).
- **NextMeso.sol**
ERC20 "NextMeso" token bridged to native currency: mint/burn tied to native deposits/withdrawals.
- **NXPCAmountManager.sol**
NXPC supply manager: enforces MAX_SUPPLY, tracks accumulated minted/burned amounts, and provides allowlisted roles for minting/burning bookkeeping.
- **CreatorWalletV2.sol**
Concrete creator multisig wallet contract with ERC2771 meta tx support which creates an initialized, self-call-restricted creator wallet.
- **DAppRewardAllocationWallet.sol**
DApp reward wallet: controlled wallet for distributing rewards to users and returning tokens to creators.
- **CreatorTokenControllerUpgradeable.sol**
Creator token controller (upgradeable): governs withdrawable vs non-withdrawable ERC20 balances for creators and provides internal allocation/withdraw helpers.

Privileged roles

The owner of the `CreatorFactory` can:

- allocate the rewards via the `allocateReward()` and `allocateRewardBatch()` function.
- set the owner of the dapps via the `setDAppOwner()` function.
- pause/unpause the contract via the `pause()` / `unpause()` functions.
- grant and revoke executors via the `grantExecutor()` and `revokeExecutor()` functions.
- call the functions with the `whenExecutable` modifier.

The executor of the `CreatorFactory`, `ItemIssuance`, `DAppRewardAllocationWallet`, `NextMeso` can:

- pause the contract via the `pause()` function.

- call the functions with the `whenExecutable` modifier when the contract is not in the pausing state.

The owner of the `ItemIssuance` can:

- create a new universe via the `createUniverse()` function.
- set a new blackhole address via the `setBlackhole()` function.
- register and unregister item721 contracts via the `registerItem721Contract()/unregisterItem721Contract()` functions.
- pause/unpause the contract via the `pause()` / `unpause()` functions.
- grant and revoke executors via the `grantExecutor()` and `revokeExecutor()` functions.
- call the functions with the `whenExecutable` modifier.

The owner of the `CreatorWalletV2` can:

- the multi-signature wallet owner can allow and call the contract's functionality with the `isSelfCall` modifier.
- call all of the functionality with the `onlyAtLeastExecutor` modifier.

The transaction requester in the `CreatorWalletV2` can call all of the functionality with the `onlyRequester` modifier.

The executor of the `CreatorWalletV2` can:

- call all of the functions with the `onlyAtLeastExecutor` modifier.

The owner of the `DAppRewardAllocationWallet` can:

- call the functions with the `whenExecutable` modifier.
- pause/unpause the contract via the `pause()` / `unpause()` functions.
- grant and revoke executors via the `grantExecutor()` and `revokeExecutor()` functions.

The owner of the `NXPAmountManager` can:

- update the operator's burn allowlisted status via the `setBurnAllowlist()` function.
- update the operator's mint allowlisted status via the `setMintAllowlist()` function.
- pause/unpause the contract via the `pause()` / `unpause()` functions.
- grant and revoke executors via the `grantExecutor()` and `revokeExecutor()` functions.

The `onlyBurnAllowlisted` role of the `NXPAmountManager` can:

- add accumulated burned amount via the `addBurnedAmount()` function.

The `onlyMintAllowlisted` role of the `NXPAmountManager` can:

- add accumulated minted amount via the `addMintedAmount()` function.

The executor of the `NXPAmountManager` can:

- pause the contract via the `pause()` function.

The owner of the `NextMeso` can:

- retrieve the Neso tokens via the `retrieveNeso()` function.
- grant and revoke executors via the `grantExecutor()` and `revokeExecutor()` functions.
- pause/unpause the contract via the `pause()` / `unpause()` functions.
- call the functions with the `whenExecutable` modifier.

Potential Risks

- The project is fully or partially centralized, introducing single points of failure and control. This centralization can lead to vulnerabilities in decision-making and operational processes, making the system more susceptible to targeted attacks or manipulation. Administrative roles retain sweeping authority, including pausing, seizing, reallocating, and upgrading contracts. This creates a strong centralization risk where compromise or misuse of a single privileged key could endanger the system.
- The audit does not cover all code in the repository. Contracts outside the audit scope may introduce vulnerabilities, potentially impacting the overall security due to the interconnected nature of smart contracts.
- The project heavily depends on off-chain processes (e.g., issuance and supply tracking), meaning the correctness of balances and allocations cannot be independently verified on-chain. Any inconsistency in the off-chain system may directly impact users without recourse.
- Arithmetic and accounting mechanisms for tokens and items (e.g., `basketAmount`, `itemAmount`, `exchangeRate`, `requireNXP`) rely on strict external assumptions. If these assumptions are violated, users may encounter locked funds, misaligned supply numbers, or unexpected behavior without any protective checks on-chain.
- The project's contracts are upgradable, allowing the administrator to update the contract logic at any time. While this provides flexibility in addressing issues and evolving the project, it also introduces risks if upgrade processes are not properly managed or secured, potentially allowing for unauthorized changes that could compromise the project's integrity and security. The beacon proxy architecture allows global upgrades of all creator wallets. A malicious or erroneous beacon implementation upgrade could compromise all deployed wallets simultaneously.

Findings

Vulnerability Details

F-2025-12421 - Missing Price Validation in `confirmRequest` - Medium

Description:

The `ItemIssuance` contract allows users to request items by sending NXP, where the price is determined by the ratio of available pool items and total supply. The helper function `expectAmount()` defines this logic:

```
function expectAmount(uint256 universe, uint256 amount)
    public view whenUniverseExists(universe) returns (uint256)
{
    uint256 itemAmount = universes[universe - 1].poolAmount;
    require(amount <= itemAmount, "ItemIssuance/invalidAmount: too large amount");

    return Math.ceilDiv(NXPAmountManager.totalSupply() * amount, itemAmount);
}
```

However, when confirming requests inside `confirmRequest()`, the contract does not validate that the requester actually paid this required NXP amount. This creates an economic vulnerability: a user can underpay during `requestItemIssuance()`, and the contract owner (or any executor with rights) can still confirm the request without verifying the correct pricing.

This breaks the intended burn-to-mint mechanics, undermines fairness between participants, and may allow malicious actors to obtain items at a discounted rate.

Assets:

- `contracts/ItemIssuance/ItemIssuance.sol`
[<https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet>]

Status:

Accepted

Classification

Impact Rate:	2/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: Introduce a validation step in `confirmRequest()` to ensure that the user paid the expected NXP value:

```
if (isConfirmed) {  
    uint256 expected = expectAmount(universe, request.itemAmount);  
    require(request.nxpAmount == expected, "ItemIssuance/underpaidRequest: insufficient NXP");  
  
    request.status = Status.CONFIRMED;  
    ...  
}
```

This guarantees that all confirmed requests respect the required pricing model, preventing underpayment exploits and preserving the intended economic integrity of the system.

Resolution: This issue has been acknowledged and accepted by the Nexpace team. The team clarified that the affected contracts have already been deployed, and therefore no code-level changes can be applied at this stage.

[F-2025-12422](#) - Risk of Locked NXP in `requestItemIssuance` - Medium

Description: The current request flow in `ItemIssuance` exposes users to the risk of permanently locked NXP. When a user initiates a request via `requestItemIssuance()`, they provide NXP (`msg.value`) which is stored within the contract:

```
function requestItemIssuance(
    uint256 universe,
    uint256 itemAmount,
    uint256 basketAmount
) external payable whenUniverseExists(universe) {
    uint256 nxpcAmount = msg.value;
    require(nxpcAmount > 0, "ItemIssuance/invalidAmount: zero value is not allowed");
    ...
}

function confirmRequest(
    uint256 universe,
    uint256 requestId,
    bool isConfirmed
) external whenUniverseExists(universe) whenExecutable {
    Request storage request = requests[requestId];

    require(request.status == Status.REQUESTED, "ItemIssuance/invalidStatus: already confirmed");
    require(request.universe == universe, "ItemIssuance/invalidRequest: universe id doesn't match");

    address target;

    if (isConfirmed) {
        request.status = Status.CONFIRMED;
        target = blackhole;

        INXPAmountManager(NXPAmountManager).addBurnedAmount(request.nxpAmount);
        universes[universe - 1].poolAmount -= request.itemAmount;

        emit RequestConfirmed(universe, requestId);
    } else {
        request.status = Status.REJECTED;
        target = request.requester;
    }
}
```

```

        emit RequestRejected(universe, requestId);
    }

    Address.sendValue(payable(target), request.nxpcAmount);
}

```

However, the NXP is **never returned or processed** unless an authority later calls `confirmRequest()`. If the authority fails to act (either by negligence, error, or malicious intent), the funds remain trapped in the contract indefinitely.

Even when `confirmRequest()` is called and `isConfirmed == true`, the requester only burns NXP and reduces the item pool, without receiving any tangible benefit (no ERC721 item transfer is triggered). This leaves users paying NXP without guaranteed reward delivery, creating an **unfair and potentially exploitable trust dependency**.

Assets:

- contracts/ItemIssuance/ItemIssuance.sol
[<https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet>]

Status:

Accepted

Classification

Impact Rate:	4/5
Likelihood Rate:	3/5
Exploitability:	Semi-Dependent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation:

Timeout mechanism: Introduce a mechanism where a request can be canceled or reclaimed if it is not processed within a certain time frame.

Enforce reward delivery: When confirming a request, ensure that the corresponding ERC721 item (or other asset) is actually issued to the requester, rather than only reducing the item pool and burning NXP.

Resolution:

This issue has been acknowledged and accepted by the Nexpace team. The team clarified that the affected contracts have already been deployed, and therefore no code-level changes can be applied at this stage.

Evidences

POC

Reproduce:

```
it.only("User loses money", async () => {
  const [, executor, , blackhole] = await ethers.getSigners();
  const { itemIssuance, request } = await loadFixture(requestItemIssuanceFixture);
  const prevBlackholeBalance = await blackhole.getBalance();

  expect(request.nxpcAmount).to.equal(expectAmount(thousand, await itemIssuance.itemPoolAmount(universeId)));
  await expect(itemIssuance.connect(executor).confirmRequest(request.universe, request.id, true)).to.be.emit(
    itemIssuance,
    "RequestConfirmed"
  );
  const afterRequest = await itemIssuance.requests(1);
  expect(await blackhole.getBalance()).to.equal(prevBlackholeBalance.add(request.nxpcAmount));
  expect(afterRequest.status).to.equal(RequestStatus.CONFIRMED);
});
```

Results:

```
~/dev/nexpace-contracts (develop) x) $ npx hardhat test
```

```
ItemIssuance Contract
  ItemIssuance Cycle
    confirmItem function
      ✓ User loses money
1 passing (652ms)
```

[F-2025-12553](#) - Unrestricted Owner Authority in retrieveNeso() Enables Direct Token Transfers - Medium

Description:

The `NextMeso` contract implements an ERC-20 token with extended functionality, including a privileged method `retrieveNeso()` designed for the owner. This function allows the owner to transfer tokens from any user address to any other address without the consent of the token holder. While this may have been introduced for administrative recovery purposes, it introduces a centralization and trust risk, as the owner effectively has unilateral control over all user funds.

Specifically, the vulnerable function is:

```
retrieveNeso(address from, address to, uint256 amount, string memory reason)
onlyOwner
```

```
/// @notice Retrieve neso by owner.1
/// @param from Address of the neso owner.
/// @param to Address of the neso receiver.
/// @param amount Amount of neso to retrieve.
/// @param reason Reason of the retrieval.
function retrieveNeso(address from, address to, uint256 amount, string memory reason) external onlyOwner {
    _transfer(from, to, amount);
    emit RetrievedNeso(from, to, amount, reason);
}
```

The root cause lies in unrestricted owner authority combined with direct token transfer from arbitrary addresses, bypassing all standard ERC-20 allowance or approval checks. This creates a scenario where token holders have no guarantee of control over their balances.

Assets:

- `NextMeso.sol` [<https://github.com/NEXPACE-Limited/maple-contract/tree/develop#>]

Status:

Accepted

Classification

Impact Rate: 5/5

Likelihood Rate: 3/5

Exploitability: Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: It is recommended to:

- remove or restrict the `retrieveNeso()` functionality. If absolutely necessary, introduce strict governance measures such as multi-signature authorization, or time-locked execution for such sensitive operations.
- add user consent checks. Implement a mechanism where token holders must pre-authorize recovery actions, or enforce an opt-in compliance model.
- introduce rate limiting. Require cool-down periods and upper limits for recovery transfers to prevent large-scale abuse.

Resolution: This issue has been acknowledged and accepted by the Nexpace team. The team clarified that the affected contracts have already been deployed, and therefore no code-level changes can be applied at this stage.

F-2025-12424 - Unsafe and Centralized Blackhole Handling - Low

Description:

The `ItemIssuance` contract relies on a configurable `blackhole` address for handling NXPC payments when requests are confirmed:

```
if (isConfirmed) {
    request.status = Status.CONFIRMED;
    target = blackhole;

    INXPAmountManager(NXPAmountManager).addBurnedAmount(request.nxpcAmount)
;

    universes[universe - 1].poolAmount -= request.itemAmount;

    emit RequestConfirmed(universe, requestId);
}
...
Address.sendValue(payable(target), request.nxpcAmount);
```

By design, a “blackhole” should behave as a **burn sink**, similar to `address(0)` or another irrecoverable destination. However, in this implementation:

- The `blackhole` is fully configurable by the contract owner via `setBlackhole()`.
- It can be set to **any address, including the owner’s wallet**.
- NXPC sent to this address will **not be burned**, but instead remain spendable within the ecosystem.

This undermines the burn mechanism, as tokens intended to be permanently removed from circulation may instead be redirected to a privileged party, **breaking economic guarantees** and enabling abuse.

Assets:

- `contracts/ItemIssuance/ItemIssuance.sol`
[<https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet>]

Status:

Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 5/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: **Immutable Blackhole:** Set `blackhole` permanently to `address(0)` (or another verifiable burn sink) at deployment and remove the ability to update it.

If configurability is required (e.g. to support upgradeability or external burn vaults), enforce strict validation:

- Require that the new address is either `address(0)` or a contract explicitly designed to burn funds.
- Add clear documentation and events to make all changes transparent.

By ensuring the blackhole truly acts as a burn mechanism, the system can preserve its economic integrity and user trust.

```
address public constant BLACKHOLE = address(0);
```

Resolution: This issue has been acknowledged and accepted by the Nexpace team. The team clarified that the affected contracts have already been deployed, and therefore no code-level changes can be applied at this stage.

[F-2025-12490](#) - Missing Validation for exchangeRate Can Lead To DOS - Low

Description:

The `constructor()` function of the `NextMeso` contract accepts `exchangeRate_` without validation and assigns it directly to the immutable `exchangeRate`. It can be seen in the code snippet below:

```
constructor(  
    address trustedForwarder,  
    ApproveController controller_,  
    uint256 exchangeRate_  
) ERC2771Context(trustedForwarder) ApproveControlled(controller_) {  
    exchangeRate = exchangeRate_;  
}
```

If `exchangeRate` is set to `0` (misconfiguration) or to an extreme value, core math in `deposit()` and `withdraw()` becomes unsafe:

- zero leads to division by zero in `withdraw()` (hard DoS) and to zero mints in `deposit()`.
- oversized values can cause frequent overflows/reverts during `deposit()` due to the following formula:
`msg.value * exchangeRate`.

Assets:

- `NextMeso.sol` [<https://github.com/NEXPACE-Limited/maple-contract/tree/develop#>]

Status:

Accepted

Classification

Impact Rate:	3/5
Likelihood Rate:	2/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

It is recommended:

- to introduce a zero validation check in the `constructor()` function for the `exchangeRate_` value:

```
if (exchangeRate_ == 0) revert ExchangeRateZero();
```

- to introduce a cap value for the `exchangeRate_` value to avoid overflow/underflow bugs.

Resolution:

This issue has been acknowledged and accepted by the Nexpace team. The team clarified that the affected contracts have already been deployed, and therefore no code-level changes can be applied at this stage.

[F-2025-12423](#) - Misleading Naming of `\_dAppOwner` - Info

Description:

The variable `_dAppOwner` in `CreatorFactory` is used to assign ownership of new `DAppRewardAllocationWallet` instances at the time of their creation:

```
address private _dAppOwner;

...

function addDApp(
    uint32 creatorId_,
    string calldata name,
    address executor
) external whenExecutable whenCreatorExists(creatorId_) {
    DAppRewardAllocationWallet dApp_ = new DAppRewardAllocationWallet(_truste
dForwarder);
    dApp_.grantExecutor(executor);
    dApp_.transferOwnership(_dAppOwner); // <- here
    ...
}
```

While the name `_dAppOwner` suggests it directly **owns all dApps** or has privileged control over them, this is not accurate. Its sole role is to act as the **initial owner** of newly deployed dApps. After deployment, control lies with that owner, and the factory does not enforce any continuing relationship.

Assets:

- `contracts/Creator/CreatorFactory.sol` [<https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet>]

Status:

Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Rename `_dAppOwner` to something more explicit, e.g. `defaultDAppOwner` or `initialDAppOwner`.

Alternatively, **improve documentation** to clearly state:

- This variable is only used as the owner assigned to new dApps.
- It does not represent global ownership or authority over existing dApps.

This will improve clarity, prevent misunderstandings, and reduce the risk of incorrect assumptions in future code usage or audits.

Resolution:

This issue has been acknowledged and accepted by the Nexpace team. The team clarified that the affected contracts have already been deployed, and therefore no code-level changes can be applied at this stage.

F-2025-12425 - Misleading Error Message in confirmRequest - Info

Description:

The `confirmRequest()` function inside `ItemIssuance` enforces that a request can only be processed if its status is still `REQUESTED`. This is done with the following check:

```
require(  
    request.status == Status.REQUESTED,  
    "ItemIssuance/invalidStatus: already confirmed"  
);
```

However, the error message `"already confirmed"` is **misleading**. The requirement will fail not only if the request has already been confirmed, but also if it has been rejected. In its current form, users and integrators might wrongly assume that the only reason for failure is confirmation, while in reality, the request might have been rejected earlier as well.

This creates ambiguity during debugging, testing, and integration, making it more difficult for users or external systems to handle failures correctly.

Assets:

- `contracts/ItemIssuance/ItemIssuance.sol`
[<https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet>]

Status:

Accepted

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

- Update the error string to reflect the actual logic being enforced. For example:

```
require(  
  request.status == Status.REQUESTED,  
  "ItemIssuance/invalidStatus: not in REQUESTED state"  
);
```

This ensures the revert reason is accurate and covers both possible invalid states (`CONFIRMED` and `REJECTED`). Clearer error messaging improves developer experience, reduces misunderstandings, and strengthens contract maintainability.

- In addition to that, consider using custom error messages to save gas.

Resolution:

This issue has been acknowledged and accepted by the Nexpace team. The team clarified that the affected contracts have already been deployed, and therefore no code-level changes can be applied at this stage.

F-2025-12426 - Redundant ERC20 Transfer Logic - Info

Description:

In `CreatorTokenControllerUpgradeable`, both `withdrawERC20()` and `allocateERC20()` functions perform identical logic — transferring ERC20 tokens from the creator wallet to a target account.

```
function _withdrawERC20(IERC20 token, address account, uint256 amount) internal {
    token.safeTransfer(account, amount);
}

function _allocateERC20(IERC20 token, address account, uint256 amount) internal {
    token.safeTransfer(account, amount);
}
```

Despite different names, both functions **do the same thing**. This duplication increases code surface without adding meaningful differentiation. The semantic distinction between "withdraw" and "allocate" is already clear at the higher-level function calls (`withdrawERC20()` vs `allocateERC20()`), so the internal implementations do not need to be separate.

Assets:

- `contracts/Creator/utis/CreatorTokenControllerUpgradeable.sol`
[<https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Unify both functions into a single internal helper (e.g. `_transferERC20()`) and call it from `withdrawERC20()` and `allocateERC20()`.

Example refactor:

```
function _transferERC20(IERC20 token, address account, uint256 amount) internal {
    token.safeTransfer(account, amount);
}

function _withdrawERC20(IERC20 token, address account, uint256 amount) internal {
    _transferERC20(token, account, amount);
}

function _allocateERC20(IERC20 token, address account, uint256 amount) internal {
    _transferERC20(token, account, amount);
}
```

This reduces code duplication, improves maintainability, and makes it explicit that both operations share the exact same underlying transfer behavior.

Resolution:

This finding is fixed by the Nexon team by creating new common function `f` or transferring token and using it in commit hash:

`2e8cd4967e98233f792f72004c7926578877d482`.

[F-2025-12452](#) - Missing Existence Guards - Info

Description:

The mutators `setCreatorName()` and `setDAppName()` index storage arrays directly using `id - 1` without validating the IDs via the existing guard modifiers (`whenCreatorExists` / `whenDAppExists`). Supplying an invalid `id` (e.g., `0` or greater than the current length) will trigger a low-level array out-of-bounds panic instead of reverting with the contract's canonical, descriptive error messages.

```
function setCreatorName(uint32 id, string calldata newName) external whenExecutable {
    emit CreatorNameChanged(id, _creators[id - 1].name, newName);
    _creators[id - 1].name = newName;
}

function setDAppName(uint32 id, string calldata newName) external whenExecutable {
    emit DAppNameChanged(id, _dApps[id - 1].name, newName);
    _dApps[id - 1].name = newName;
}
```

Assets:

- contracts/Creator/CreatorFactory.sol [<https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet>]

Status:

Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Enforce ID validity using the existing guard modifiers to align with the rest of the contract and provide clear revert reasons.

```

function setCreatorName(uint32 id, string calldata newName)
    external
    whenExecutable
    whenCreatorExists(id)
{
    ...
function setDAppName(uint32 id, string calldata newName)
    external
    whenExecutable
    whenDAppExists(id)
{

```

Resolution:

This issue has been acknowledged and accepted by the Nexpace team. The team clarified that the affected contracts have already been deployed, and therefore no code-level changes can be applied at this stage.

[F-2025-12486](#) - Excessive Revert String Length Increases Gas Costs and Deployment Size - Info

Description:

Several contracts across the codebase use long revert strings in `require` statements, such as:

the `onlyOwnedDApp` modifier in the `CreatorWalletLogicUpgradeableV2` contract:

```
modifier onlyOwnedDApp(uint32 dAppId) {
    require(
        creatorFactory.creatorIdOfDApp(dAppId) == creatorId(),
        "CreatorWalletLogicUpgradeable/forbidden: The dApp is not owned b
y the creator"
    );
    require(
        creatorFactory.isActiveDApp(dAppId),
        "CreatorWalletLogicUpgradeable/inactiveDApp: given an inactive id
"
    );
    _;
}
```

These verbose messages significantly increase gas costs for reverted transactions and inflate the deployed bytecode size, ultimately resulting in wasted funds for both deployers and end users.

Assets:

- `contracts/Creator/CreatorFactory.sol` [<https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet>]
- `contracts/Creator/CreatorWallet/CreatorWalletLogicUpgradeableV2.sol` [<https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet>]
- `contracts/Creator/CreatorWallet/CreatorWalletV2.sol` [<https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet>]
- `contracts/ItemIssuance/ItemIssuance.sol` [<https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet>]
- `contracts/NXPC/NXPCAmountManager.sol` [<https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet>]
- `NextMeso.sol` [<https://github.com/NEXPACE-Limited/maple-contract/tree/develop#>]

Status: Accepted

Classification

Impact Rate: 1/5
Likelihood Rate: 1/5
Exploitability: Independent
Complexity: Simple
Severity: Info

Recommendations

Remediation: It is recommended to replace long error messages with the custom error types, which are far more gas-efficient than string-based reverts. For instance:

```
error NotDAppOwner(uint32 dAppId, uint256 creatorId);  
error InactiveDApp(uint32 dAppId);  
  
modifier onlyOwnedDApp(uint32 dAppId) {  
    uint256 ownerId = creatorFactory.creatorIdOfDApp(dAppId);  
    if (ownerId != creatorId()) revert NotDAppOwner(dAppId, ownerId);  
    if (!creatorFactory.isActiveDApp(dAppId)) revert InactiveDApp(dAppId);  
    _;  
}
```

Resolution: This issue has been acknowledged and accepted by the Nexpace team. The team clarified that they would adhere to their own internal message error convention and no code-level changes will be applied at this stage.

[F-2025-12585](#) - Missing Validation for poolAmount > 0 in requestItemIssuance() Allows Zero ItemAmount Requests - Info

Description: The `ItemIssuance.sol` contract contains the `requestItemIssuance()` function, which processes item issuance requests by recording them and incrementing the value of the `requestLength`. It can be seen in the code snippet below:

```
function requestItemIssuance(
    uint256 universe,
    uint256 itemAmount,
    uint256 basketAmount
) external payable whenUniverseExists(universe) {
    uint256 nxpcAmount = msg.value;
    address requester = _msgSender();

    require(universes[universe - 1].poolAmount >= itemAmount, "ItemIssuance/invalidAmount: too large amount");
    require(nxpcAmount > 0, "ItemIssuance/invalidAmount: zero value is not allowed");

    requestLength++;

    requests[requestLength] = Request({
        universe: universe,
        id: requestLength,
        requester: requester,
        itemAmount: itemAmount,
        basketAmount: basketAmount,
        nxpcAmount: nxpcAmount,
        status: Status.REQUESTED
    });

    emit ItemRequested(universe, requestLength, requester, itemAmount, basketAmount, nxpcAmount);
}
```

However, the function does not validate that the `poolAmount` is greater than zero, allowing a user to submit requests with `itemAmount = 0` and `msg.value = 1` intentionally or by mistake when the `poolAmount` is zero, which will eventually increment the value of the `requestLength` and potentially give no value in return to the user.

Assets:

- contracts/ItemIssuance/ItemIssuance.sol
[<https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet>]

Status:**Accepted****Classification****Impact Rate:** 1/5**Likelihood Rate:** 1/5**Exploitability:** Independent**Complexity:** Simple**Severity:** Info**Recommendations****Remediation:**

Add a validation to the `requestItemIssuance()` function to ensure `poolAmount` is strictly positive:

```
require(poolAmount > 0, "Pool amount must be greater than zero");
```

Resolution:

This issue has been acknowledged and accepted by the Nexpace team. The team clarified that the affected contracts have already been deployed, and therefore no code-level changes can be applied at this stage.

F-2025-12615 - Missing Events For Critical Data - Info

Description:

Several sensitive functions in the `NXPAmountManager` contract suite modify contract state or make critical changes without emitting events. Events are essential in blockchain applications to track state changes, especially in sensitive operations such as updating addresses, permissions, or configurations. Without these event emissions, it becomes difficult to track contract actions, which can hinder transparency and potentially obscure malicious or erroneous changes.

`NXPAmountManager`

```
function setMintAllowlist(address operator, bool allowlisted) external onlyOwner{}  
function setBurnAllowlist(address operator, bool allowlisted) external onlyOwner{}
```

External observers (e.g., dApps, trackers) cannot easily ascertain when specific state changes occur without events. Off-chain systems, which might be reacting to contract state changes, may not be able to detect those changes, leading to discrepancies.

Assets:

- contracts/NXPC/NXPAmountManager.sol
[<https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet>]

Status:

Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

To enhance transparency and traceability, it is recommended to emit events in key functions. This will allow users and external services to monitor and react to changes. Ensure that every critical action, especially those modifying contract states or handling funds, emits an event.

Resolution:

This issue has been acknowledged and accepted by the Nexpace team. The team clarified that the affected contracts have already been deployed, and therefore no code-level changes can be applied at this stage.

[F-2025-12623](#) - Manipulation in expectAmount() Can Mislead Off-Chain Pricing Logic - Info

Description:

The `ItemIssuance::expectAmount()` function calculates the required NXP amount for item issuance based on the conceptual supply returned by `NXPAmountManager::totalSupply()` and the universe's `poolAmount`. However, if the conceptual supply becomes zero (e.g., due to governance-influenced burned counters or incorrect configuration), the formula in the code snippet below will return 0 for any non-zero amount:

```
function expectAmount(uint256 universe, uint256 amount) public view whenUniverseExists(universe) returns (uint256) {
    uint256 itemAmount = universes[universe - 1].poolAmount;

    require(amount <= itemAmount, "ItemIssuance/invalidAmount: too large amount");

    return Math.ceilDiv(NXPAmountManager.totalSupply() * amount, itemAmount);
}
```

This problem can arise because the calculation depends on the `NXPAmountManager::totalSupply()` function, which is conceptual and can be driven to zero by governance-controlled functionality `NXPAmountManager::addBurnedAmount()` without any actual token burns.

```
function totalSupply() public view returns (uint256) {
    return MAX_SUPPLY + _accumulatedMintedAmount - _accumulatedBurnedAmount;
}
```

On the other hand, the value of the `poolAmount` can also be manipulated and set significantly high via the `ItemIssuance::addItem()` function, which can be seen in the code snippet below:

```
function addItem(uint256 universe, uint256 amount) external whenUniverseExists(universe) whenExecutable {
    universes[universe - 1].poolAmount += amount;
    emit ItemAdded(universe, amount);
}
```

While the `ItemIssuance::requestItemIssuance()` function enforces `msg.value > 0`, external systems, frontends or users might rely on `expectAmount()` for pricing estimations before sending transactions. Returning 0 can

therefore mislead off-chain integrators into assuming issuance is free, resulting in broken UX or failed transactions.

Assets:

- contracts/ItemIssuance/ItemIssuance.sol
[<https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet>]
- contracts/NXPC/NXPCAmountManager.sol
[<https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet>]

Status:

Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations**Remediation:**

It is recommended to introduce a validation check in the `expectAmount()` function to ensure that the product of the `totalSupply() * amount` is greater or equal to `itemAmount`, which is the `poolAmount` of a specific universe. Moreover, it is advisable to introduce a cap limit for the `poolAmount` value that can be increased via the `addItem()` function as well as the maximum amount of native tokens that can be burned via the `addBurnedAmount()` function.

Resolution:

This issue has been acknowledged and accepted by the Nexpace team. The team clarified that the affected contracts have already been deployed, and therefore no code-level changes can be applied at this stage.

[F-2025-12633](#) - Incomplete Function Logic Leads to Discrepancy Between Implementation and Developer Intentions - Info

Description:

The `addItem721BaseAmount()` function of the `ItemIssuance` contract is declared with a developer note suggesting that it increases the base amount of an ERC721 item. However, the implementation merely emits an event without modifying any state or performing the intended logic. It can be seen in the code snippet below:

```
/// @notice Increases the base amount of an item within the specified universe for ERC721 tokens
/// @param itemId The ID of the item to increase the base amount for
/// @param newLimitSupply The amount to add to the base amount of the item
function addItem721BaseAmount(
    uint256 itemId,
    uint256 newLimitSupply
) external whenUniverseExists(universeOfItem721[_msgSender()]) {
    emit ItemBaseAmountAdded(universeOfItem721[_msgSender()], _msgSender(), itemId, newLimitSupply);
}
```

Similar problem exists within the `allocateItems()` function of the `CreatorWalletLogicUpgradeableV2` contract:

```
/// @notice Records the allocation of items with their hashed data to creator-owned dapp accounts
/// @param dAppId A id of dApp
/// @param hashedData A hashed data of items
function allocateItems(uint32 dAppId, bytes memory hashedData) external isSelfCall onlyOwnedDApp(dAppId) {
    emit ItemsAllocated(dAppId, hashedData);
}
```

This creates a mismatch between the documentation and the behavior of the contract. Such incomplete logic can lead to confusion for developers, integrators, and auditors who rely on NatSpec comments and emitted events to understand the function's effects.

Assets:

- `contracts/Creator/CreatorWallet/CreatorWalletLogicUpgradeableV2.sol` [<https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet>]
- `contracts/ItemIssuance/ItemIssuance.sol` [<https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet>]

Status:

Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity: Info

Recommendations

Remediation: Consider either updating the documentation to reflect the function's limited purpose including the note about the off-chain part of the process or implement the full behavior as documented.

Resolution: This issue has been acknowledged and accepted by the Nexpace team. The team clarified that the affected contracts have already been deployed, and therefore no code-level changes can be applied at this stage.

[F-2025-12639](#) - Checks-Effects-Interactions Pattern Violation - Info

Description:

State variables are updated after the external calls to the `CreatorWalletV2` and `DAppRewardAllocationWallet` contracts.

As explained in [Solidity Security Considerations](#), it is best practice to follow the [checks-effects-interactions pattern](#) when interacting with external contracts to avoid reentrancy-related issues.

The following assets are affected by this pattern violation:

`CreatorFactory::addCreator()`

```
function addCreator(string calldata name, address[] memory owners, uint256 threshold) external whenExecutable {
    address account = MinProxy.createBeaconProxy(creatorBeacon);
    CreatorWallet(payable(account)).initialize(owners, threshold);
    _creators.push(Creator({ name: name, account: account }));
    _creatorId[address(account)] = uint32(_creators.length);

    emit CreatorAdded(uint32(_creators.length), name, account);
}
```

`CreatorFactory::addDApp()`

```
function addDApp(
    uint32 creatorId_,
    string calldata name,
    address executor
) external whenExecutable whenCreatorExists(creatorId_) {
    DAppRewardAllocationWallet dApp_ = new DAppRewardAllocationWallet(_trustedForwarder);
    dApp_.grantExecutor(executor);
    dApp_.transferOwnership(_dAppOwner);
    _dApps.push(DApp({ creatorId: creatorId_, isActive: true, name: name, account: address(dApp_) }));
    _dAppId[address(dApp_)] = uint32(_dApps.length);

    emit DAppAdded(uint32(_dApps.length), name, address(dApp_));
}
```

Assets:

- `contracts/Creator/CreatorFactory.sol` [<https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet>]

Status: Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Follow the [checks-effects-interactions pattern](#) when interacting with external contracts. Conduct all external interactions after updates to the state. If this approach is not possible, consider using a [reentrancy guard](#).

Resolution: This issue has been acknowledged and accepted by the Nexpace team. The team clarified that the affected contracts have already been deployed, and therefore no code-level changes can be applied at this stage.

[F-2025-12641](#) - Missing Address Zero Validation - Info

Description: The project's functionality lack validation checks for zero addresses. Specifically:
The function `setDAppOwner()` from the `CreatorFactory` contract lacks address zero validation for the `newDAppOwner` parameter:

```
function setDAppOwner(address newDAppOwner) external onlyOwner {  
    emit DAppOwnerChanged(_dAppOwner, newDAppOwner);  
    _dAppOwner = newDAppOwner;  
}
```

Assets:

- contracts/Creator/CreatorFactory.sol [<https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet>]

Status: Accepted

Classification

Impact Rate: 1/5
Likelihood Rate: 1/5
Exploitability: Dependent
Complexity: Simple
Severity: Info

Recommendations

Remediation: It is recommended to implement a zero validation check for the `newDAppOwner` parameter in the `CreatorFactory::setDAppOwner()`.

Resolution: This issue has been acknowledged and accepted by the Nexpace team. The team clarified that the affected contracts have already been deployed, and therefore no code-level changes can be applied at this stage.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository1	https://github.com/NEXPACE-Limited/nexpace-contracts
Commit	acf6c488040a992e6a0e98e68a8d7081bfb036fe
Remediation Commit for Repository1	2e8cd4967e98233f792f72004c7926578877d482
Repository2	https://github.com/NEXPACE-Limited/maple-contract
Commit	acfc6421409675ea36952bf87996c9b1a5daf2d7
Whitepaper	N/A
Requirements	NatSpec
Technical Requirements	NatSpec

Asset	Type
contracts/Creator/CreatorFactory.sol [https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet]	Smart Contract
contracts/Creator/CreatorWallet/CreatorWalletLogicUpgradeableV2.sol [https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet]	Smart Contract
contracts/Creator/CreatorWallet/CreatorWalletV2.sol [https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet]	Smart Contract
contracts/Creator/DAppRewardAllocationWallet/DAppRewardAllocationWallet.sol [https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet]	Smart Contract
contracts/Creator/Utils/CreatorTokenControllerUpgradeable.sol [https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet]	Smart Contract
contracts/ItemIssuance/ItemIssuance.sol [https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet]	Smart Contract
contracts/NXPC/NXPCAmountManager.sol [https://github.com/NEXPACE-Limited/nexpace-contracts/tree/feature/upgrdae-creator-wallet]	Smart Contract
NextMeso.sol [https://github.com/NEXPACE-Limited/maple-contract/tree/develop#]	Smart Contract

Appendix 3. Additional Valuables

Verification of System Invariants

During the audit of **NEXSPACE LIMITED/ Creator Wallet**, Hacken followed its methodology by performing fuzz-testing on the project's main functions. Foundry, a tool used for fuzz-testing, was employed to check how the protocol behaves under various inputs. Due to the complex and dynamic interactions within the protocol, unexpected edge cases might arise. Therefore, it was important to use fuzz-testing to ensure that several system invariants hold true in all situations.

Fuzz-testing allows the input of many random data points into the system, helping to identify issues that regular testing might miss. A specific Echidna fuzzing suite was prepared for this task, and throughout the assessment, **3** invariants were tested over **20,000** runs. This thorough testing ensured that the system works correctly even with unexpected or unusual inputs.

Invariant	Test Result	Run Count
<code>NextMeso::withdraw()</code> should always allow to withdraw the full amount of native currency a user deposited	Passed	20000
<code>NextMeso::withdraw()</code> should always allow to withdraw the remaining deposited native currency if a user withdrew a portion of it	Passed	20000
<code>ItemIssuance::expectAmount()</code> should always return non-zero amount for non-zero inputs	Passed	20000

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.